

Cloud Based Secure Banking Application

Neha Raghoba Parab
Information Technology, PCCE,
parabneha8993@gmail.com

Louella M. Mesquita Colaco
Computer Department, PCCE,
Imesquita@rediffmail.com

Fiona Coutinho
Computer Department, PCCE,
coutinho.fiona@gmail.com

Abstract— Individual Banks store vast amounts of user sensitive data on their private server. Sharing this data with other banks will provide an efficient way of offering their customers with personalized services and will deliver value-added data services to the entire banking sector. This data is highly sensitive and requires a secure sharing medium. However, secure data sharing is a taxing and troublesome job. This paper proposes a framework for sharing Secure Sensitive Data between banks which includes Secure Data Transfer, Secure Data Storage, Secure Data Delivery and Usage & Destruction of Data which uses Variable AES in place of Normal AES for securely storing the data as Variable AES is an enhancement to Normal AES since it uses new sub key for each block of data. This framework provides security of the user's sensitive data and shares this data safely. To allow banks to share their data we have implemented this framework using the cloud application where we provide user authentication, data integrity and access control using the Proxy Re-Encryption mechanism. At the same time, data owners (Bank) retain complete control of their own data in a safe and sound environment.

Keywords—Banking Application, Cloud Platform, Proxy Re-Encryption, Variable AES

I. Introduction

In today's digital era, each organization wants to store their customer information in a secure and safe manner. The stored information not only meets the demand of the organization itself but can also meet the demands of other organizations, if stored on a global platform. In a banking organization the data comprises of its customer's private information, but if this data can be shared with other banks, it can benefit the functioning of the entire banking sector. Individual banks generate and maintain information of their customers. This information is only available to the respective bank.

Consider the following scenario:

A customer goes to apply for a loan at Bank A. He/She takes a loan for a certain amount, for a specific period of time. This loan details are only available to Bank A.

After a period of time the same customer needs another loan, but now he/she applies for a loan in Bank B. Bank B needs to learn the creditworthiness of the customer in order to sanction the loan. If the banks don't share the data/loan information of its customer then it will affect the calculation of the creditworthiness of the loan applicant.

But if both banks agree to share their data, Bank B can retrieve Bank A's data about the customer and be in a better position to decide whether the loan application for the said amount is to be processed and sanctioned or rejected.

We can conclude that enforcing the system mentioned above in real life scenarios, can give rise to the following issues:

- Security: In today's world keeping sensitive data in plain form is vulnerable to several security breaches.
- Restriction: Giving other organizations direct access to one's database is a big security issue.
- Distribution: Sharing data with all organization violates customer's privacy.

In this paper we propose a framework where individual banks can share their customer details with other banks, in a secure and easy manner. We allow banks to share their customer's data with other trusted banks by giving the owner bank control to decide which other bank they want to share their data with. We make use of a cloud application that has access to sharable data. Other banks will use this cloud application to access the details for a particular customer. We use different mechanisms for securing data transmission and destruction of the data after its use on the cloud platform.

II. Literature Review

Secure data transmission is a process of transferring sensitive data securely from a source to destination. Work in [1], shows the development of a systematic secure sensitive data sharing framework which ensures security of user sensitive data and at the same time provides the data owner with complete control of their data. It guarantees secure use of clear text in cloud platform. This frameworks uses Advanced Encryption Standard (AES) algorithm for data encryption and access control strategy which requires the data owner to Re-Encrypt the data based on Proxy Re-Encryption (PRE) mechanism. The goal of PRE is to securely enable the re-encryption of cipher text from one secret key to another, without relying on third parties. Introduced by Blaze, Bleumer & Strauss, in 1998, PRE mechanism was the first bidirectional PRE scheme. They proposed an application called atomic proxy re-encryption in which semi trusted proxy converts cipher for user A to cipher which can be decrypted by User B. The work in [4] shows different Proxy Re-Encryption methods based on bilinear maps. To minimize user's secret key storage [4] presents a method based on the El Gamal scheme which operates over two groups of prime order. Another improvement to PRE is mentioned in [6]. It is a Time based Proxy Re-Encryption mechanism. It uses the idea of cipher text policy attribute based encryption, which identifies a set of attributes for each user and generates a user attribute secret key (UAK). The data owner outsources the data to the cloud service provider (CSP). When the user wants to access the data, it first requests the data owner to send the appropriate key and then requests the CSP to download the data. The UAK for each user depends on user attribute and time period for those attributes, which denotes the validity of the user. Hence

the user can only access the data in the given time period. The main disadvantage of this scheme is user revocation as the data owner needs to be online to submit the UAK to the requesting user. An improvement to this challenge is stated in [2] which enables cloud based user revocation. It ensures low computation and low communication overhead on the data owner as the entire processing (distributing keys and encrypted data) is handled by the cloud platform.

According to [1] to keep our data confidential it is important to encrypt the data with AES. AES is considered the most secure encryption mechanism in current time, but there are many who think that the only point of failure in AES is the key. Analysis on AES in [3] categorizes two attacks on it based on its key. Work in [3] shows a way to further secure AES using Variable key cipher design. Where it combines the model of block cipher and stream cipher to increase the security of the cipher. Variable Key Cipher is a method in which we generate sub keys from original key to encrypt the block of text. There are two methods used to generate sub keys in Paper [3]: AES with feedback and AES with counter. Experimental results show that the security of AES is increased using variable AES as it makes the prediction of keys difficult. To enhance the AES encryption algorithm further and to achieve various security goals model in [7] proposes a new symmetric algorithm called as an E-AES by adding a new step which includes logical XOR operation which ensures improving AES security in terms of avalanche effect. Here the plaintext and key are both converted into binary code. As AES is a symmetric encryption technique, we need the same secret key to encrypt as well as decrypt the data. Quantum key distribution (QKD) model in [8] proposes a technique of AES key distribution using the Quantum channels. QAES combines the AES technique and Quantum key distribution without affecting the security of AES algorithm. Here the secret key is generated over the quantum channel and given to sender and receiver depending on their online availability. Once the keys are received, the sender uses a different key for each 128 bit of data which is generated by QKD in particular rounds.

III. Cloud based secure banking application

To transfer sensitive data from one bank to another, we need a full secure cycle with reliable submission, storage, usage and destruction of the data after use. We are required to deal with three main issues- security of the data entered, the restriction on the access to this data and the distribution of the data to trusted recipients only. Sharing sensitive information over the network gives rise to the need of security. Encryption technique changes the original data into an encoded form using a secret key, which can be decrypted back into original data only by the person who has access to the same secret key. We will be using Variable AES to encrypt our data. Variable AES [3] is an improvement over Normal AES which uses different keys to encrypt each 128bit block of data.

Giving another organization the complete access to the owner's database makes the database vulnerable and increases the chances of attacks on it. We need to give access rights to other organization for sharable data only, and not to the entire database. Hence we will make use of a shared cloud platform where we can keep the sharable data. The cloud platform will

act as a provider of data to the bank that has access rights to the data shared by the owner bank. Hence the data will only be available to the banks that are trusted by the owner bank.

We assume that we cannot completely trust the cloud platform, hence we keep the data in encrypted form on the cloud platform. As we need to maintain the secrecy of the cipher key used in encrypting the data, we advise encryption on the cipher key using public key of the data owner as it can only be decrypted by the valid private key of the data owner, which in turn can be used to decrypt the customer's sensitive information. In our scenario we need the requesting bank to be able to decrypt the encrypted cipher key which is encrypted using public key of the owner bank. There are two methods: First, we can let the owner bank share his private key with the cloud platform that can decrypt the cipher key and give it to the requesting bank. According to previous assumption, cloud platform cannot be completely trusted, hence giving private key to cloud platform to decrypt the cipher key might result in a security breach. But still we want the other bank to be able to decrypt the data. Hence, we need to convert the encrypted data in such a way that it can be decrypted using requesting banks private key. To ensure secure storage and access control we employ the second method which is to use Proxy Re-Encryption technique. Proxy Re-Encryption ensures that the data (in our case cipher key) remains in encrypted form on the semi trusted cloud platform and only the bank that has valid access right can access the data. The Entire System Architecture for our secure banking application is depicted in Fig.1.

A. Data Submission

When the owner Bank wants to store their customer information, it opens the Bank application and enters the sensitive private information. On the submission of the data, the application fetches a copy of the PAN Card for reference and begins the encryption process. Encryption ensures confidentiality of the information. In our application we will use Symmetric and Asymmetric encryption techniques in a hybrid manner. We will encrypt the Sensitive data using symmetric key and transmit this key using asymmetric encryption technique. After validation of the data by the bank application, the process of encryption starts by generating a secret 32 byte key, which is submitted to Variable AES [3] that encrypts the data. On the completion of the encryption mechanism, the application takes the secret key and encrypts it using Proxy Re-Encryption Method [2],[4],[6] which uses the Public key of the owner bank and the global values retrieved from the global database, present on the cloud, which is shared by all the banks.

The banking application then sends a copy of the PAN card of the entered customer along with its encrypted data file and encrypted secret key to the local database of the bank as well as to the cloud database. The data stored on the cloud platform are the records that the owner bank is willing to share with other banks but the data is encrypted with symmetric encryption method and symmetric key is encrypted using Asymmetric technique, hence it can be decrypted only with valid private key. To avoid the risk of sharing the private key with other banks, the owner bank will use the Proxy key generation technique to produce PRE Keys for the Banks it is

willing to share the data with and stores it on the cloud platform in advance. The owner Bank generates PRE Keys for

the banks by using his own private Key and the other bank's Public key, which he has retrieved from the cloud platform.

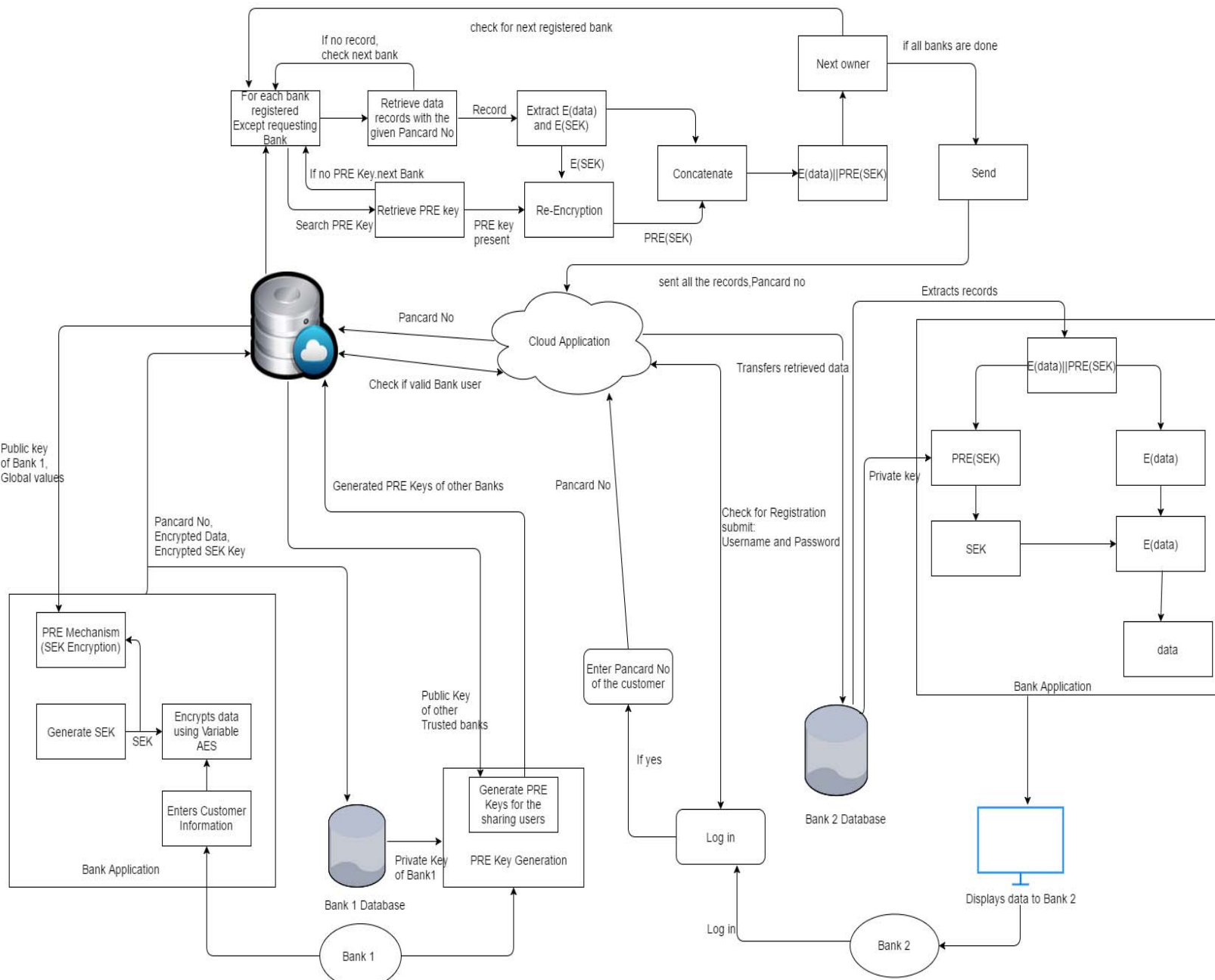


Fig.1. Cloud Based Secure Banking Application System Architecture

B. Data Retrieval

When a Bank wants to check if a customer has been sanctioned loans in other banks, it logs on to the cloud application. If the bank is a registered user of cloud application, it prompts the user bank to enter the customers PAN card details. The cloud application checks the database of all the registered banks based on the entered PAN Card details. If any bank has an entry for that PAN Card, it checks if there is PRE key for the requesting bank by the owner bank. If the PRE key is present, it retrieves the PRE key and the record of that PAN Card. It extracts the encrypted key from that record and uses the PRE Re-Encryption mechanism. It

then uses the global values and the retrieved PRE Key to produce a new re-encrypted key. If no PRE key is found for the requesting bank than the application continues its search for the PAN Card in other registered banks. Proxy Re-Encryption [2],[4],[6] is a technique which converts a cipher text which can be decrypted by owner banks private key into a cipher text that now can be decrypted by requesting banks private key. The application then transfers the retrieved record with this re-encrypted key back to the requesting Banks database. The application continues this process for all the banks apart from the requesting Bank. Once the cloud application ends, the bank application starts. It checks if the

local database has a new entry from the cloud platform, if so it extracts the record if any (PAN Card, encrypted data and Re encrypted secret key). The re-encrypted secret key now can be decrypted by private key of the requesting Bank. The PRE re-encryption technique on cloud changes the cipher text that could be decrypted only by owner banks private key in such a way that now it can be decrypted by requesting banks private key without compromising the data. Once the bank application has recovered the secret key, it uses this secret key on the encrypted data using decryption function of Variable AES to get back the private customer information. After the local application decrypts the records sent by the cloud platform it searches for any records present in its local database for the same PAN Card. If found it uses its own private key to get back the details. It displays data from all the sources to the bank. Once the bank closes the application all the record of the data from the local database is deleted.

IV. System Requirements

A. Hardware Requirements

We have implemented the Local Bank application on a System having 2.20 GHz CPU and 8GB RAM. The data stored on the cloud platform (Google Cloud) machine, has 2 GHZ CPU and 7.5 GB RAM each.

B. Software Requirements

The Bank application is implemented as a dynamic Web Application on Eclipse Mars Release 4.5.0, which is being executed on the Apache Tomcat Server 7.0.75 version. The Banking application takes the PRE mechanism which is implemented as a Python 3.6 program for supporting large value calculations. MySQL 5[10] is used to store data on the local server. The cloud platform used for the application is Google cloud [9] which uses the cloud SQL 5.6 or 5.7 [9],[10] to store sharable data sent by the bank.

V. Implementation

Banking Application needs data to be secure, through experimental observation conducted on our data set we can show that Variable AES is better than Normal AES in terms of security. But the time taken by Variable AES is approx. twice the time taken by Normal AES (refer Table I and Table II). To implement the above application we have used one application for encryption and other application for decryption. Both the applications are implemented using Eclipse IDE which are running on a Tomcat Server. To facilitate our scenario we have generated a file from all the data entered in the application. We will avoid encrypting a single attribute data of the customer at a time as it will increase the number of time we call the encryption algorithm and the storage call to the database. In our banking scenario, the encryption application will have two classes: Normal AES which will be used to generate the sub keys and the other class will be the Variable AES that encrypts the data. To generate the sub keys the application will first need to know the number of sub keys to be produce. For this we will divide the length of the file by 128 bit .The resultant number will be the amount of sub keys that has to be produced. We use random number generator

function to generate a 256 bit real key. Random number generator function is used to produce a random string of data hence guessing the real key will be difficult. Once the real key is generated we will use Normal AES to generate the first sub key using the real key as plaintext and encrypting it with real key which will also act as key for encryption. After the first sub key is produced we will XOR this key with the real key to produce the new plaintext .This new plaintext is again encrypted using real key to produce next sub key. We will continue this process till required number of sub keys are generated. We will store this new keys in a dynamic keylist. After the sub keys are produce we call the Variable AES algorithm. The difference between Normal AES and Variable AES in our scenario is that in Normal AES we allow key expansion before dividing the data in 128 bit block as same key has to be used whereas in Variable AES we will first let the algorithm divide the data in 128 bit block and then retrieve each block. Depending on the block number the sub key is retrieved from keylist, which is than given to key expansion function after which the encryption algorithm starts. This is done for each 128 bit block. The same method of key generation is used in our decryption application as well. To incorporate and fit to our banking application we have divided our PRE mechanism in 3 stages:

1. We will use encryption stage of PRE at our encryption application to encrypt the SEK key. The SEK key is encrypted using public key of the owner bank and random values.
2. We will use the Re-Encryption stage on the cloud platform to re-encrypt the SEK key again with the PRE Key which is generated in advance by owner bank for trusted banks.
3. We will use the Decryption stage in our decryption application to decrypt the re-encrypted SEK key with the requesting banks private key.

For our application we have implemented PRE mechanism using Python programming language as Java don't support exponential calculation of floating point number also data structure in Java cant store the larger values generated by PRE mechanism. Python is user friendly and it also don't require explicit declaration of variables.

We have used Google Cloud as our cloud platform which runs our cloud application. Google Cloud is easy to use and also supports Java and Python programming languages.

We allow individual banks to store their data in individual SQL instances which can be accessed by that bank and the cloud application. SQL instances act as a private bank server placed on a shared platform.

In our application framework we are submitting the data to the local database of the requesting bank. This is done to ensure that the requesting bank don't have to share its private key with the cloud platform. If we give cloud platform private key details it might modify the data which will comprise customer's private and sensitive information.

In our application the data records are only transferred when a bank with valid access right try's to request the data. Hence the scenario of the data being send to a non-trusting bank is almost null. If by any chance a non-trusting bank masquerades

a valid bank, it won't be able to decrypt the data as it won't have valid private key of the real bank.

VI. Result

Implementing the above application on two Banks, we have been able to retrieve all the records present for a customer in both the banks hence validating the above framework.

The application was validated using positive cases and negative cases. In positive cases we were successful in retrieving the accurate result whereas for negative cases we can infer that the framework don't allow unauthorized access to the shared data. Hence we can concluded that the above framework is secure to transfer sensitive data.

We have stated earlier than Variable AES is better than Normal AES, we have proved it experimentally by using it on our application. Table I shows the comparison among Normal AES and Variable AES based on our application. Table II shows the experimental comparison between Normal AES and Variable AES based on time taken for encryption and decryption separately for our application.

TABLE I. OBSERVATIONAL DIFFERENCE BETWEEN NORMAL AES AND VARIABLE AES

	Normal AES	Variable AES
Algorithm to generate Real key	Random Key Generation	Random Key Generation
No of keys used	1	Depends on size of the plaintext(size in bytes/16bytes)
Key sizes	128/192/256	128/192/256
Algorithm used to generate variable key	-	AES
Key used for each block	Same Real key	Different keys(1 key for each block)
Time taken	T ms	Approx. 2T ms(& more)
Attacks possible	Attack with same key	Not possible

TABLE II. EXPERIMENTAL RESULTS ON TIMING DIFFERENCE BETWEEN NORMAL AES VS VARIABLE

	Normal AES		Variable AES		
Key Length	Encrypt time	Decrypt time	Key Generation	Encrypt time	Decrypt time
AES 128	8ms	4ms	13ms	8ms	4ms
AES 192	10ms	4ms	14ms	7ms	6ms
AES 256	11ms	5ms	20ms	10ms	7ms

From TABLE I we can note that Variable AES can be implemented in any of the key size depending on the user requirement. The number of sub keys generated in Variable AES depends on the total size of the actual plaintext from whom we are generating the sub keys. The time estimation stated in TABLE I can be proved by the values seen in TABLE II. TABLE II gives the timing estimate of Variable AES and Normal AES for encryption and decryption of data. As the key size increases the time taken by the AES cipher increases. From TABLE II we can conclude that Variable AES take approx. twice the time required by the Normal AES.

VII. Conclusion

We have described and implemented a Cloud based secure banking application framework that provides data integrity by encrypting the sensitive data during its transfer and storage, it also provides user authentication as only bank with valid access rights can access this data .This framework supports data destruction as data is destroyed when the application closes. As the data is kept in encrypted form we also provide security against semi trusted platform. Future enhancement can be done on python which we used to implement our PRE mechanism as Python provides a limited support for exponential calculation of larger floating point numbers.

References

- [1] Xinhua Dong, Ruixuan Li , Heng He, Wanwan Zhou, Zhengyuan Xue, and Hao Wu, "Secure Sensitive Data Sharing on a Big Data Platform," TSINGHUA SCIENCE AND TECHNOLOGY ISSN 1007-0214 08/11 pp72-80 Volume 20, Number 1, February 2015.
- [2] Yanjiang Yang,Haiyan Zhu,Haibing Lu,Jian Weng,Youcheng Zhang,Kim-Kwang Raymond Choo, "Cloud based data sharing with fine grained proxy re-encryption.Pervasive and Mobile Computing," Elsevier Science Publishers pp122-134 Volume 28 Issue C, June 2016.
- [3] Shaaban Sahmoud, Wisam Elmasry and Shadi Abudalfa, "Enhancement the Security of AES Against Modern Attacks by Using Variable Key Block Cipher," International Arab Journal of e-Technology, Volume 3, Number 1, January 2013.
- [4] Giuseppe Ateniese, Kevin Fu, Matthew Green, Susan Hohenberger, " Improved Proxy Re-Encryption Schemes with Applications to Secure Distributed Storage," ACM Transactions on Information and System Security(TISSEC) Volume 9 Issue 1, February 2006.
- [5] Behrouz A. Forouzan , Debdeep Mukhopadhyay , "Cryptography and Network Security," McGrawHill 2nd Edition ISBN-13:978-0-07-070208-0, isbn-10:0-07-070208-X.
- [6] Qin Liu,Guojun Wang,Jie Wu, "Time-based proxy re-encryption scheme for secure data sharing in a cloud environment," Information Sciences: an International Journal pp355-370 Volume 258, February 2014 .
- [7] Ajeet Singh, "A New Approach to Enhance Avalanche Effect in Aes to Improve Computer Security," Journal of Information Technology & Software Engineering ISSN: 2165-7866.April 2015
- [8] Omer K.Jasim Mohammad, Safia Abbas, El-Sayed M.El-Horbaty ,Abdel-Badeeh M.Salem, "Innovative Method for enhancing Key generation and Management in the AES-algorithm".
- [9] "Google Cloud Platform," September 2014 [Online].Available: <https://cloud.google.com/docs/>
- [10] "MySQL Documentation," October 2015 [Online]. Available: <https://dev.mysql.com/docs/>
- [11] Simon Heron, "Advanced Encryption Standard (AES)," Elsevier Science Publishers B. V. Amsterdam, The Netherlands, The Netherlands pp 8-12. Volume 2009 Issue 12, December ,2009